

ATIVIDADE PRÁTICA

Pipeline de Dados com Python

Extensão com Geolocalização, Airflow e Git

Extração · DW · Data Lake · Data Mart · Dashboard · Docker · Airflow · Git

Documento	Demanda de Negócio — Atividade Prática
Referência	DEM-BI-2024-014-EXT
Baseado em	DEM-BI-2024-014 (Aula 14 — desenvolvido em sala)
Entrega	Link do repositório Git enviado ao professor
Formação	Python para Dados — Digital College · Atividade Pós Aula 14

⚠ LEIA ANTES DE COMEÇAR

Esta atividade parte do projeto desenvolvido em sala na Aula 14. Antes de começar, certifique-se de que o projeto base está funcionando corretamente em sua máquina: conexão com o PostgreSQL, carga no Redshift, HDFS disponível, Data Mart criado e dashboard rodando na porta 8050.

1. Contexto

O pipeline construído em sala (DEM-BI-2024-014) entregou com sucesso a cadeia completa de dados: do PostgreSQL transacional ao dashboard interativo com Dash/Plotly, passando por Redshift, HDFS e Data Mart. A Diretoria de Business Intelligence aprovou a solução e, com base nos primeiros resultados, identificou uma nova necessidade de negócio: incorporar a dimensão geográfica das vendas — estado e cidade de destino — para permitir análise regional da performance comercial.

Paralelamente, a área de TI apontou dois gaps de maturidade na solução entregue:

- Ausência de orquestração: o pipeline é executado manualmente via notebook, sem agendamento, sem logs centralizados e sem tratamento de falhas por etapa.
- Ausência de versionamento: o código não está em um repositório Git, impossibilitando rastreabilidade de mudanças, colaboração entre desenvolvedores e integração com pipelines de CI/CD futuros.

Esta atividade propõe a extensão do projeto para resolver os dois pontos acima.

2. Problema de Negócio

A equipe comercial precisa responder às seguintes perguntas que o pipeline atual não consegue responder:

Pergunta de negócio	Por que o pipeline atual não responde?
Quais estados concentram o maior volume de vendas?	A fato_venda não possui a dimensão geográfica do cliente.
Quais cidades têm maior receita real vs. esperada?	Cidade e estado não foram extraídos do schema geral.
Existe concentração regional nas vendas acima da meta?	Sem dim_localidade não é possível cruzar atingimento por região.
O pipeline rodou hoje? Em qual etapa falhou?	Não há orquestração; o notebook é executado manualmente.
Quem alterou o código do dashboard e quando?	Sem Git não há histórico de mudanças rastreável.

3. Objetivos da Extensão

3.1 Objetivo Geral

Estender o pipeline da Aula 14 com três incrementos: (1) inclusão da dimensão geográfica estado/cidade no DW e no dashboard; (2) orquestração automática de todas as etapas com Apache Airflow; (3) versionamento completo do projeto no Git com boas práticas de repositório.

3.2 Objetivos Específicos

- Extrair do schema geral as tabelas geral.endereco, geral.cidade e geral.estado para compor a nova dimensão dim_localidade.
- Enriquecer a fato_venda com a sk_localidade, associando cada venda ao endereço de entrega do cliente.
- Adicionar ao Data Mart a sumarização por estado e cidade (tabela vendas_localidade).
- Incluir no dashboard dois novos gráficos: ranking de receita por estado e mapa de calor de atingimento por cidade.
- Criar uma DAG no Apache Airflow que execute as 6 etapas do pipeline de forma sequencial e agendada.
- Publicar o projeto completo em um repositório Git (GitHub, GitLab ou Bitbucket) com estrutura organizada e README detalhado.

4. Escopo da Atividade

DENTRO DO ESCOPO	FORA DO ESCOPO
Extração de geral.endereco, geral.cidade e geral.estado	Geolocalização por coordenadas (lat/lon)
Criação de dim_localidade no Redshift	Análise de rotas ou logística de entrega
Enriquecimento da fato_venda com sk_localidade	Integração com APIs de CEP externas
Tabela vendas_localidade no Data Mart	Autenticação no dashboard
Criar novos gráficos no novo dashboard (estado e cidade)	Deploy em cloud pública (AWS, GCP, Azure)
DAG Airflow com as 6 etapas + etapas novas	Testes automatizados (pytest)
Repositório Git com README, .gitignore e .env.example	CI/CD (GitHub Actions, etc.)

5. Extensão do Modelo de Dados

5.1 Nova Dimensão — dim_localidade

A dimensão geográfica deve ser construída a partir do JOIN entre as tabelas geral.endereco, geral.cidade e geral.estado. Cada linha representa uma combinação única de endereço de entrega vinculado a uma pessoa (id_pessoa).

Coluna	Tipo	Origem	Descrição
sk_localidade	integer	Gerada pelo pipeline	Surrogate key da dimensão
id_pessoa	integer	geral.endereco	FK para geral.pessoa (chave de junção com a fato)
cidade	text	geral.cidade	Nome da cidade de entrega
estado	text	geral.estado	Descrição completa do estado
sigla_estado	text	geral.estado	Sigla UF (ex: CE, SP, RJ)
bairro	text	geral.bairro	Bairro do endereço (opcional, pode ser nulo)
cep	text	geral.endereco	CEP do endereço de entrega

Query sugerida para extração (ponto de partida — o aluno deve adaptar conforme necessário):

```
SELECT
    e.id_pessoa,
    c.descricao AS cidade,
    est.descricao AS estado,
    est.sigla AS sigla_estado,
    b.descricao AS bairro,
    e.cep
FROM geral.endereco e
JOIN geral.cidade c ON c.id = e.id_cidade
JOIN geral.estado est ON est.id = c.id_estado
LEFT JOIN geral.bairro b ON b.id = e.id_bairro;
```

5.2 Enriquecimento da fato_venda

A tabela `fato_venda` já existente deve receber a coluna `sk_localidade`. O vínculo é feito através do `id_cliente` da nota fiscal, que corresponde ao `id_pessoa` em `geral.endereco`:

- Fazer o merge entre `df_fato` e `dim_localidade` usando `id_cliente ↔ id_pessoa`.
- Validar nulos em `sk_localidade`: clientes sem endereço cadastrado gerarão valores nulos (documentar a quantidade encontrada).
- Aplicar o mesmo padrão `TRUNCATE + INSERT` da `fato` já existente no Redshift.

5.3 Nova tabela no Data Mart — `vendas_localidade`

O Data Mart deve ser estendido com a tabela `vendas_localidade`, contendo a sumarização de vendas por localização:

Coluna	Tipo	Descrição
<code>sigla_estado</code>	text	UF de destino da venda (agrupador primário)
<code>estado</code>	text	Nome completo do estado
<code>cidade</code>	text	Cidade de destino da venda
<code>qtde_vendida</code>	numeric	Soma das quantidades vendidas para aquela localidade
<code>valor_total_real</code>	numeric(18,2)	Receita real acumulada para aquela localidade
<code>valor_total_esperado</code>	numeric(18,2)	Meta acumulada para aquela localidade

6. Orquestração com Apache Airflow

6.1 Contexto

O Apache Airflow é a ferramenta padrão de mercado para orquestração de pipelines de dados. Com ele, cada etapa do pipeline vira uma Task independente dentro de uma DAG (Directed Acyclic Graph). Isso traz: agendamento automático, reexecução de etapas com falha, log centralizado por task e visibilidade do histórico de execuções.

6.2 Estrutura da DAG

O aluno deve criar o arquivo `dags/pipeline_vendas.py` com uma DAG que orquestre todas as etapas do pipeline de forma sequencial. A estrutura mínima esperada é:

Ordem	task_id sugerido	O que a task deve executar
-------	------------------	----------------------------

1	extrair_banco_fonte	Conectar ao PostgreSQL e extrair todas as tabelas necessárias (incluindo geral.endereco, geral.cidade, geral.estado)
2	carregar_redshift	Criar/atualizar dim_tempo, dim_produto, dim_cliente, dim_vendedor, dim_localidade e fato_venda no Redshift
3	gravar_data_lake	Gerar Parquet particionado localmente e enviar ao HDFS
4	popular_data_mart	Ler do HDFS, agregar e gravar vendas_ano_mes e vendas_localidade no PostgreSQL DM
5	validar_pipeline	Verificar contagens esperadas em cada tabela de destino e registrar resultado no log da DAG

Dependências entre tasks (fluxo sequencial obrigatório):

```
extrair_banco_fonte >> carregar_redshift >> gravar_data_lake >> popular_data_mart  
>> validar_pipeline
```

6.3 Configurações mínimas da DAG

- dag_id: 'pipeline_vendas_digital_corporativo'
- schedule_interval: '@daily' (executa uma vez por dia, à meia-noite)
- start_date: data de início definida pelo aluno (usar dias_atrás=1)
- catchup=False: não executar runs anteriores ao deploy
- Todas as credenciais devem vir de variáveis de ambiente ou do Airflow Connections — nunca hardcoded.
- Em caso de falha, a task deve tentar novamente 1 vez com intervalo de 5 minutos (retries=1, retry_delay).

6.4 Tipo de Operator recomendado

Para as tasks de Python, utilize o PythonOperator importando as funções já criadas no projeto. Encapsule cada etapa em uma função Python separada (ex: def task_extrair(), def task_carregar_redshift()) e passe-as para o callable do PythonOperator.

Dica: use o XCom (ti.xcom_push / ti.xcom_pull) para passar o número de linhas processadas entre tasks e usar esses valores na task de validação.

7. Versionamento com Git

7.1 Estrutura esperada do repositório

O repositório deve seguir a estrutura abaixo. Itens marcados com * são novos em relação ao projeto da aula:

Arquivo / Pasta	Conteúdo esperado
README.md *	Descrição do projeto, pré-requisitos, instruções de execução e diagrama da arquitetura
.gitignore *	Ignorar: .env, __pycache__, *.pyc, lake/, node_modules/ e qualquer arquivo com credenciais
.env.example *	Template do .env com todas as variáveis necessárias, sem valores reais (ex: SOURCE_DATABASE_URL=)
analise.ipynb	Notebook de desenvolvimento com todas as etapas (incluindo as novas)
app.py	Dashboard Dash/Plotly atualizado com os 2 novos gráficos
dags/pipeline_vendas.py *	DAG do Airflow com as 5 tasks sequenciais
Dockerfile	Containerização do app.py (sem alterações obrigatórias)
docker-compose.yml	Orquestração do container do dashboard
requirements.txt	Dependências Python atualizadas (incluir apache-airflow se necessário)

7.2 Boas práticas obrigatórias

- O arquivo .env NUNCA deve ser commitado. Verificar antes de cada push.
- Usar mensagens de commit descritivas e em português ou inglês (ex: 'feat: adiciona dim_localidade ao schema estrela').
- O repositório deve ter ao menos 3 commits distintos, representando as etapas de desenvolvimento.
- O README.md deve conter: descrição do projeto, tecnologias usadas, instruções de como rodar localmente e instruções de como configurar o .env.
- Colocar o repositório como público para facilitar a avaliação pelo professor.

7.3 Commits sugeridos (mínimo)

#	Mensagem de commit sugerida
---	-----------------------------

1	init: estrutura inicial do projeto com .gitignore, .env.example e README
2	feat: adiciona extração de endereço, cidade e estado e cria dim_localidade
3	feat: enriquece fato_venda com sk_localidade e cria tabela vendas_localidade no Data Mart
4	feat: adiciona gráficos de ranking por estado e atingimento por cidade no dashboard
5	feat: cria DAG do Airflow com 5 tasks sequenciais para orquestração do pipeline
6+	fix / docs / refactor — commits adicionais à medida que o aluno evoluir

8. Extensão do Dashboard

O app.py deve ser estendido com dois novos gráficos alimentados pela tabela vendas_localidade do Data Mart. Os gráficos existentes devem ser mantidos intactos.

8.1 Gráfico 1 — Ranking de Receita por Estado

- Tipo: gráfico de barras horizontais (go.Bar com orientation='h').
- Dados: top 10 estados por valor_total_real, ordenados de maior para menor.
- Exibir junto à barra o percentual de atingimento (pct_atingimento) como anotação.
- Título: 'Receita Real por Estado (Top 10)'.

8.2 Gráfico 2 — Atingimento por Cidade (Top 20)

- Tipo: gráfico de barras verticais agrupadas (go.Bar).
- Dados: top 20 cidades por valor_total_real, exibindo lado a lado valor_total_esperado e valor_total_real.
- Adicionar linha de referência (add_hline) em 100% sobre o gráfico de atingimento.
- Título: 'Receita Real vs. Meta — Top 20 Cidades'.

8.3 Integração ao filtro existente

Os dois novos gráficos devem responder ao filtro de ano já existente. Para isso, acrescente os novos outputs ao callback atualizar() e filtre vendas_localidade pelo ano selecionado (a tabela vendas_localidade deve conter a coluna ano para viabilizar esse filtro).

9. Requisitos Técnicos

9.1 Requisitos Funcionais

- RF-06: dim_localidade criada no Redshift com sk_localidade, cidade, estado, sigla_estado, bairro e cep.
- RF-07: fato_venda enriquecida com sk_localidade no Redshift.
- RF-08: tabela vendas_localidade disponível no Data Mart com agrupamento por estado/cidade/ano.
- RF-09: dashboard exibindo ranking por estado e atingimento por cidade, respondendo ao filtro de ano.
- RF-10: DAG pipeline_vendas.py funcional no Airflow com as 5 tasks e agendamento @daily.
- RF-11: repositório Git público com estrutura completa, .gitignore correto e README detalhado.

9.2 Requisitos Não Funcionais

- RNF-06: credenciais nunca expostas no repositório Git — uso obrigatório de .env e .env.example.
- RNF-07: cada task da DAG deve ter retries=1 e registrar no log o número de linhas processadas.
- RNF-08: o dashboard deve continuar funcional com docker-compose up após as alterações.
- RNF-09: o notebook analise.ipynb deve refletir as novas etapas com células de validação (contagens e verificação de nulos).

10. Critérios de Aceite

Geolocalização

- dim_localidade criada no Redshift com pelo menos as colunas sk_localidade, id_pessoa, cidade, estado e sigla_estado.
- fato_venda possui a coluna sk_localidade; quantidade de nulos documentada no notebook.
- tabela vendas_localidade disponível no Data Mart com colunas sigla_estado, cidade, valor_total_real, valor_total_esperado, pct_atingimento e ano.

Dashboard

- Os dois novos gráficos aparecem no painel sem erros de callback.
- O filtro de ano atualiza todos os gráficos (incluindo os novos) de forma reativa.

Airflow

- A DAG aparece na interface do Airflow sem erros de importação.
- Uma execução manual (Trigger DAG) completa todas as 5 tasks com status success.
- O log de cada task exibe a contagem de linhas processadas.

Git

- O repositório é público e acessível pelo professor sem autenticação.
- O arquivo .env não consta em nenhum commit (verificar com git log --all -- .env).
- O README.md contém: descrição, tecnologias, instruções de execução e configuração do .env.
- Existem ao menos 5 commits com mensagens descritivas.

11. Entrega

Item	Detalhes
O que entregar	Link do repositório Git (GitHub, GitLab ou Bitbucket)
Onde enviar	Grupo do Whats App
Prazo	12/05/2026

DICA FINAL

Não tente construir tudo de uma vez. Siga a ordem deste documento: primeiro valide a `dim_localidade` no notebook, depois enriqueça a fato, depois o Data Mart, depois o dashboard e só então a DAG. Commite ao final de cada etapa funcional. Assim, mesmo que a DAG não fique pronta, as entregas anteriores já valem.

12. Stack Tecnológica Completa

Categoria	Ferramenta	Finalidade nesta atividade
Já visto em aula	Python · pandas · SQLAlchemy	Extração, transformação e carga
Já visto em aula	Amazon Redshift	DW com nova dim_localidade e fato enriquecida
Já visto em aula	HDFS + pyarrow (Parquet)	Data Lake — sem alterações estruturais
Já visto em aula	PostgreSQL + Dash/Plotly	Data Mart estendido + dashboard com 2 novos gráficos
Já visto em aula	Docker + docker-compose	Containerização do dashboard — sem alterações obrigatórias
Já visto em aula	Apache Airflow	Orquestração da DAG pipeline_vendas com 5 tasks sequenciais
Já visto em aula	Git + GitHub/GitLab	Versionamento, .gitignore, commits semânticos e README

Documento gerado como extensão da Aula 14 da Formação Python para Dados — Digital College. Esta atividade deve ser realizada individualmente. Plágio resultará em desqualificação da entrega.